TRAPSHIFT

ONE MORE TRY

Project Documentation Pack

Game Design • Technical Stack • Architecture • Assets • Level Plan • QA • Play Store Checklist

---

Tagline: One More Try. One More Trap.

Prepared for a new, from-scratch Android game project using Google AI Studio for AI-assisted development and Android Studio for build, testing, and release.

Version 1.0 • 8 November 2026

How to use this document pack

This PDF contains the supporting project documents and technical decisions for building TRAPSHIFT from scratch. The separate Master Development Prompt should be pasted into Google AI Studio; this PDF is the reference specification the AI agent should follow.

Recommended workflow: create the initial Android project in Google AI Studio, build a small playable prototype, export the source project, then use Android Studio to inspect, build, install, and test it. Do not attempt all 50 levels and all modes in one generation request.

Important: AI-generated code must be reviewed and tested. A successful code generation or preview is not proof that the game is complete, fair, stable, or ready for Google Play.

1. Game Overview — Game Design Document

1.1 Identity

Field	Specification
Game title	TRAPSHIFT – One More Try
Tagline	One More Try. One More Trap.
Genre	2D puzzle / arcade / trap-based challenge
Core player action	Guide a ball from its starting point to the goal while understanding and avoiding traps.
Core experience	Short, readable puzzles; surprising but fair traps; quick retries; satisfying completion.
Initial release approach	Offline-first single-player Android game; no mandatory account or backend.

1.2 Core gameplay loop

1. Select a level or game mode.
2. Read the board, visible clues, and available route.
3. Move the ball using simple, responsive controls.
4. Avoid or solve traps using logic, timing, memory, and observation.
5. Reach the destination to complete the level.
6. Earn stars/rewards, save progress, and continue or retry.

1.3 Design principles

• Fair difficulty: a player should be able to learn why a trap activates. Avoid unavoidable deaths and unexplained random punishment.

• Fast retry: failure should lead to a quick restart without forced waiting.

• Readable visuals: distinguish safe tiles, hazards, switches, exits, and active timers.

• Progressive learning: introduce one mechanic at a time before combining mechanics.

• Replayability: use meaningful puzzle variety, personal bests, and optional cosmetics—not manipulative systems.

2. Game Modes and Content Plan

2.1 Level Mode — 50 handcrafted levels

Level range	Learning / challenge focus
1–5	Movement, boundaries, reaching the goal, basic instructions
6–10	Basic hazards, safe routes, simple trap clues
11–20	Moving obstacles, disappearing floors, countdown tiles
21–30	Fake exits, portals, switches, bridges, shifting walls
31–40	Reverse controls, memory traps, invisible paths, combinations
41–50	Advanced multi-step puzzles and carefully tested combinations

Each level must have at least one valid solution, a clear start and goal, correct boundaries, and a reliable reset state. Keep a level metadata record containing level ID, board dimensions, mechanic list, difficulty, intended solution or validation data, and completion criteria.

2.2 Endless Mode

- Generate progressively harder board challenges using controlled difficulty bands.
- Validate generated layouts before presenting them; reject unreachable goals and impossible trap combinations.
- Save the local personal best and session statistics.
- Provide immediate retry and a clear game-over reason.

2.3 Daily Challenge

- Generate one deterministic puzzle per calendar date using a documented seed algorithm and game-generation version.
- The same date and generation version should produce the same puzzle on different devices.
- Save local completion history and streak information.
- Do not promise global leaderboards or cross-device synchronization without a backend.
- Recognize that a fully offline app cannot securely prove the real date. Prevent casual duplicate reward exploits, document limitations, and avoid claiming tamper-proof streaks.

2.4 Rage Mode

- Use deliberately tricky but solvable levels, limited moves, fake exits, shifting floors, reverse controls, and combinations.
- Make surprises legible through clues, animation, sound, or consistent rules.
- Do not use random unavoidable failures or deceptive controls that make the game feel broken.

3. Trap Mechanics Specification

Mechanic	Expected behaviour and fairness requirement
Fake safe tile	Looks safe initially but activates under a learnable condition; provide a clue before the player can reasonably infer the rule.
Shifting wall	Changes position at a defined trigger or interval; never closes every valid route without warning.
Invisible path	Revealed by a clue, switch, scan-like interaction, or logical action; indicate where the player should investigate.
Memory trap	Uses the player's earlier moves/actions in the current puzzle; clearly define what is remembered and reset it on restart.

Mechanic	Expected behaviour and fairness requirement
Timed obstacle	Uses visible countdown or predictable cycle; never activates earlier than displayed.
Moving obstacle	Has predictable movement/collision rules and clear visual state.
Disappearing floor	Show warning or countdown before removal; reset consistently.
Fake exit	Clearly part of a puzzle clue; avoid arbitrary goal switching without feedback.
Reverse controls	Signal activation and duration; provide clear feedback and restore controls correctly.
Portal	Define entry/exit, direction, and collision behaviour; prevent loops or invalid destinations.
Switch/bridge	Switch state must visibly update bridge state; state must reset correctly.
Combination trap	Combine previously taught mechanics; verify at least one safe solution.

3.1 Required trap lifecycle

1. Idle: trap is inactive and its state is known to the game engine.
2. Warning: if the mechanic needs warning, show its clue or countdown.
3. Triggered: apply the defined effect exactly once or according to documented repeat rules.
4. Resolved: return to a safe state, remain active, or become disabled as designed.
5. Reset: restore all level-specific trap state when the player restarts.

4. Recommended Technology Stack

For the first version, use a native Android implementation because the target is an Android app and the intended board is a relatively simple 2D puzzle scene. Keep the actual game rules independent from the UI so the rendering layer can be changed later if needed.

Technology	Purpose	Recommendation
Kotlin	Main Android programming language	Use
Jetpack Compose	Home, menus, dialogs, settings, HUD, navigation	Use
Compose Canvas	Draw the 2D grid, ball, tiles, traps, paths, and effects	Use for initial board renderer
Android Studio	Open project, inspect code, build, run, debug, create APK/AAB	Required for verification
Room	Persist levels, progress, rewards, statistics, daily history	Use if structured local data is needed
DataStore	Persist settings such as sound, music, vibration, theme	Use
Kotlin Coroutines / Flow	Async work and observable state	Use
JUnit	Unit tests for game rules, movement, trap logic, level validation	Use
Compose UI Test	Test navigation and UI interactions	Use where practical
Gradle Kotlin DSL	Build configuration and dependency management	Use project defaults where possible

4.1 Optional alternatives

Godot 4: a strong option if the project later requires a dedicated 2D game engine, physics-heavy scenes, tile maps, and animation tooling. However, do not mix Godot and a native Compose game implementation in the same first prototype without a clear reason. Pick one primary game runtime.

Google AI Studio: use it as an AI-assisted coding environment to generate and revise the Android project. It is not a substitute for Android Studio build verification, real-device testing, signing, or Play Console checks.

4.2 Avoid unnecessary dependencies

- Do not add a backend, account system, ads SDK, analytics SDK, or payment SDK to the initial offline release unless the product scope changes.
- Do not add large game frameworks for simple grid movement without measuring the need.
- Keep dependency versions compatible with the generated project and current Android tooling.

5. Suggested Project Architecture

Keep gameplay rules in a plain Kotlin game/domain layer. Compose should display state and send player actions; it should not contain the authoritative trap or level rules.

Layer / package	Responsibility
app	Application entry point, navigation, app-level dependency setup
core/model	Position, direction, tile type, level definition, game state, result models
core/engine	Movement rules, collision checks, turn resolution, win/fail conditions
core/traps	Trap interfaces, trap state, trigger and reset behaviour
core/validation	Level reachability checks and procedural layout validation
data/local	Room database, entities, DAOs, DataStore preferences
data/repository	Progress, settings, rewards, daily challenge persistence
feature/home	Home screen and mode selection
feature/levels	Level map, lock/unlock and selection
feature/gameplay	Board rendering, input handling, HUD, pause, result screens
feature/endless	Procedural run setup and personal best
feature/daily	Date seed, daily puzzle and local history
feature/rage	Rage level rules and limited-move handling
feature/stats	Statistics and achievements
feature/settings	Audio, vibration, accessibility and preferences
assets	Original/licensed images, fonts, audio, icons and level data

The package names are a suggested organization, not a mandate to rebuild an existing generated structure. Keep the actual project simple and consistent.

6. Controls, Screens and User Experience

6.1 Controls

- Choose one primary control scheme for the prototype: directional buttons, swipe movement, or tap-to-move.
- Test controls on a real phone; avoid accidental diagonal or repeated movement unless explicitly supported.
- Provide clear input feedback and prevent input from continuing after pause, failure, or level completion.

- Make movement speed and animation duration consistent with the game rules.

6.2 Required screens

Screen	Main elements
Home	Game title, Play, Level Mode, Endless, Daily Challenge, Rage Mode, Statistics, Settings
Level select	Level grid, locked/unlocked state, stars, current progress
Gameplay	Board, ball, goal, move count, timer if needed, pause, restart, hint
Pause	Resume, restart, settings shortcut, home
Failure	Reason/message, retry, restart, home
Victory	Stars, moves, reward, next level, replay
Settings	Music, sound effects, vibration, accessibility options

6.3 Accessibility and usability

- Do not communicate hazards only through color; use shape, icon, animation, or labels as well.
- Keep text readable on common phone sizes.
- Provide independent music and sound-effect toggles and optional vibration.
- Avoid intense flashing effects and excessive screen shake.
- Test portrait layout first unless landscape is deliberately selected as the game's standard.

7. Art, Audio and Asset Checklist

Asset	Initial need	Notes
App icon	Required before release	Original, readable at small sizes
Ball/player graphic	Prototype can use a vector shape	Create a consistent base style and optional skins
Goal graphic	Required	Visually distinct from traps
Tile and wall graphics	Required	Safe, blocked, hidden, warning, active states
Trap graphics	Required for implemented mechanics	Use clear states and consistent visual language
UI icons	Required for polished UI	Play, pause, retry, hint, settings, sound
Backgrounds	Optional	Keep contrast and readability high
Animations	Recommended	Movement, trigger, warning, victory, failure
Sound effects	Recommended	Movement, trap, collision, success, failure, buttons
Background music	Optional	Loopable, original or properly licensed

Asset rule: use original assets, self-created vector art, or assets with licenses that explicitly allow commercial game use. Keep a record of asset source, license, author, and attribution requirements. Do not copy another game's art, logo, level layouts, music, or distinctive UI.

8. Data and Save-System Requirements

8.1 Data to persist locally

- Unlocked and completed levels.
- Stars and best move count per level.
- Coins and cosmetic unlocks.
- Endless personal best and selected statistics.
- Daily challenge completion history and local streak.
- Achievements and player preferences.

8.2 Data safety rules

- Save progress at safe points and handle app process death.
- Use database migrations if the schema changes after release.
- Do not reset user progress when adding new levels or features.
- Avoid duplicate reward grants when a result screen is reopened.
- Document offline daily-date limitations; do not claim secure global verification.

9. Quality Assurance and Test Plan

Test area	Acceptance criteria
Movement	No movement through walls; inputs are responsive; no unwanted movement after pause/end.
Collision	Ball and hazard interactions match documented rules.
Trap timing	No timed trap activates before its visible countdown or defined cycle.
Trap reset	Restart restores every trap, switch, floor, portal, timer, and memory state.
Level validity	Every handcrafted level has a verified route and reachable goal.
Procedural generation	Generated layouts pass reachability and fairness validation before play.
Progress saving	Progress survives app close/reopen and device rotation where applicable.
Rewards	No duplicate coin, star, or completion grants from repeated callbacks.
Daily challenge	Same date/version yields the same puzzle; local history behaves consistently.
UI	All visible controls work; no clipping on supported screen sizes.
Performance	No persistent frame drops, memory leaks, or excessive battery use during normal play.
Build	Debug build installs and launches; release build is generated and tested before publishing.

9.1 Release blockers

- Any crash on normal navigation or gameplay.
- Any level that cannot be completed or reset correctly.
- A trap that causes an unavoidable or unexplained failure.
- Data loss or duplicate reward exploit affecting progression.
- A visible button that does not work.
- A release build that has not been installed and smoke-tested.

10. Google Play Store Preparation

Prepare these items only after the game is stable and the release scope is final.

Item	What to prepare
Application ID	Unique reverse-domain package name; choose carefully before publishing.
Versioning	Version name and monotonically increasing version code.
App icon	High-quality original launcher icon meeting current Play requirements.
Store listing	Game title, short description, full description, category, contact details.
Screenshots	Real screenshots from the final game; do not depict unavailable features.
Feature graphic	Original promotional artwork meeting current Play Console dimensions.
Privacy policy	Accurately describe actual app behaviour, SDKs, data handling, and contact details.
Data Safety	Answer based on actual code and all included third-party SDKs.
Content rating	Complete the Play Console questionnaire accurately.
Target audience	Choose intended age groups honestly; Families requirements apply if children are targeted.
Target API	Meet the current Play Console target API requirement at submission time.
App signing	Use Play App Signing and securely preserve upload key/backups.
Testing track	Complete required internal/closed testing and any account-specific testing requirements.

Google Play policies and target API requirements change. Verify the current requirements in Play Console and official Android/Google Play documentation immediately before submission. Approval is not guaranteed.

11. Development Milestones and Deliverables

Milestone	Exit criteria / deliverable
M0 — Project setup	Clean Android project opens, builds, and launches.
M1 — Prototype	One complete playable level with ball movement, walls, goal, collision, win/fail, restart.
M2 — First 10 levels	10 distinct playable and verified levels; core traps behave correctly.
M3 — Full Level Mode	50 handcrafted, verified levels and persistent progression.
M4 — Additional modes	Endless, Daily Challenge, and Rage Mode each work independently.
M5 — Polish	UI, animations, sound, hints, rewards, statistics, accessibility.
M6 — QA	Automated tests, manual test report, bug fixes, performance checks.
M7 — Release candidate	Debug APK and signed AAB; release build installed and smoke-tested; store materials prepared.

12. Project Folder and File Checklist

The AI agent should create a sensible project structure and keep it consistent. This is a checklist of expected categories, not a requirement to create every file before coding.

- Project configuration: settings.gradle.kts, root build.gradle.kts, gradle.properties, app/build.gradle.kts, AndroidManifest.xml.
- Application: MainActivity, application theme, navigation setup.
- Game domain: models, board state, movement rules, collision rules, game engine, win/fail conditions.
- Traps: trap definitions, trap state handling, trigger logic, reset logic.

- Levels: level definitions/data, level repository, validation tests.
- Persistence: Room entities/DAO/database, repositories, DataStore preferences.
- Features: home, level select, gameplay, pause, result, endless, daily, rage, stats, settings.
- Assets: drawable/vector assets, fonts, audio, app icon, any license/source notes.
- Tests: unit tests for movement, collision, traps, reset, level validity, progression, daily seed, rewards; UI tests where useful.

13. What to ask the AI agent to report after each phase

1. List every file created, modified, and deleted, with a short reason.
2. Summarize what is actually implemented—not what is merely planned.
3. List the commands/build tasks and tests actually run, including their results.
4. List known bugs, limitations, and untested areas.
5. Confirm whether the app was installed and manually tested; if not, say so.
6. Provide the next recommended milestone and any decision needed from the developer.

Final principle: prioritize a genuinely fun and fair playable game over a large feature checklist. A smaller, verified prototype is a better foundation than a large collection of unfinished screens.